INFO345: Research Topics in Recommender Systems

To what extent does blending CF scores with content-based cosine scores from TF-IDF text features reduce popularity bias while improving Top-N accuracy on Amazon Books

Candidate IDs: 129, 111, 122

Submission date: November 24, 2025

Total pages: 12

This report follows the ACM sigconf format and INFO345 guidelines.

To what extent does blending CF scores with content-based cosine scores from TF-IDF text features reduce popularity bias while improving Top-N accuracy on Amazon Books

Anonymous for assessment
University of Bergen
Bergen, Norway

Abstract

Recommender systems are crucial to helping users find what they want from a large selection of items, but they often suffer from popularity bias and cold-start problems. In the Amazon Books Reviews dataset, a long-tail analysis shows that a small fraction of highly popular books account for the vast majority of interactions, indicating a strong bias toward head items. In this study, we ask: to what extent does blending collaborative filtering (CF) scores with content-based cosine scores from TF–IDF text features reduce popularity bias while maintaining strong Top-N accuracy on Amazon Books?

We construct three main recommenders: an item-based kNN CF model trained on user-item ratings, a TF-IDF content-based model using book titles and descriptions, and a late-fusion hybrid that linearly combines per-user z-normalized CF, CBF, and popularity scores. All models are evaluated offline with a held-out test split using standard Top-N ranking metrics (Precision@10, Recall@10, nDCG@10, and hit_rate@10), with popularity and random baselines providing reference floors. The CF model achieves the highest Top-10 accuracy (Precision@10 $\approx$ 0.066, Recall@10 $\approx$ 0.665), while the hybrid attains almost the same accuracy with only a marginal drop in these metrics and moderately increases catalog coverage from 0.9274 to 0.9385. However, the hybrid's novelty percentile (0.6038) is slightly lower than that of CF (0.6555), indicating it recommends more popular items on average than pure CF.

We found that item-kNN CF gets the highest overall top-10 accuracy, while the hybrid model achieves similar accuracy and recommends a broader range of items. Although catalog coverage improves, the hybrid does not reduce popularity bias as measured by novelty. The CBF model clearly outperforms the baselines and improves coverage of tail items, but remains weaker than the CF and the hybrid in terms of ranking quality. Overall, the results indicate that a simple weighted hybrid can increase catalog coverage without sacrificing Top-N accuracy, but does not consistently reduce popularity bias compared to CF.

Contents

1 Introduction

In the modern digital era, recommender systems are not just helpful, but essential. They play a major role in shaping user experiences across platforms such as online shopping, streaming services and social media. These systems act as filters for huge item inventories and thus manage the problem of information overload. They personalize content for each user to help in content discovery and considerably improve user engagement. Their economic influence is substantial because they not only help businesses in selling their products, but also in retaining their customers and making them happy through matching the users with items according to their preferences.

Recommender systems, although widely adopted, still face inherent challenges that can affect their performance and fairness. The issue of **popularity bias** and **the cold-start problem** are two of the main problems. The term popularity bias is used to describe the behavior of a number of recommendation algorithms that, while recommending items, disproportionately suggest popular items, often at the expense of less popular, niche, or "long-tail" content. This can result in a scenario where users get very similar recommendations (filter bubbles) and the selection of new interests is reduced. On the other hand, the cold-start problem occurs when new users or items lack interaction data and as a result, algorithms are unable to provide precise recommendations. Addressing these challenges is crucial for developing more robust, equitable, and user-centric recommender systems [2, 7].

Hybrid recommender systems have emerged as a promising solution that combines different recommendation algorithms as a whole to eliminate these limitations. Collaborative Filtering (CF) excels at capturing complex user-item interaction patterns but struggles with cold-start scenarios and can exacerbate popularity bias. The other method, Content-Based Filtering (CBF), on the contrary, makes use of the item metadata to recommend similar items, thus providing a natural solution to cold-start problems and even increasing the diversity of the recommended items. This study investigates a hybrid approach that blends scores derived from Collaborative Filtering with content-based cosine scores generated from TF-IDF text features, aiming to harness the complementary strengths of both methods [6, 9].

The research question guiding this study is: *To what extent does blending CF scores with content-based cosine scores from TF-IDF text features reduce popularity bias while improving Top-N accuracy on Amazon Books?* The goal of this research is to measure the effect of this hybridization strategy on the two major performance dimensions, which are the reduction of popularity bias and the increase in recommendation accuracy. We are going to use the Amazon Books dataset to provide empirical evidence and insight into the pros and cons of this blending method.

The study is structured in the following manner: Section 2 gives a thorough motivation for the research, with a focus on the importance of the two main issues, popularity bias and the cold-start problem. Section 3, reviews the relevant literature concerning the recommender system paradigms, hybridization strategies and the evaluation frameworks. The Amazon Books dataset is presented in Section 4, which includes its properties, the preprocessing phase, and the difficulties it poses. Section 5 elaborates on the methodology, while Section 6 provides a detailed description of the experimental setup, which covers the implementation and hyperparameter tuning. The experimental results are shown and interpreted in Section 7. Finally, Section 8 discusses the findings, Section 9 identifies the limitations of this research, and Section 10 addresses ethical and privacy considerations in recommender systems. Section 11 details the study's reproducibility aspects. Section 12 wraps up the study with a summarization of our findings and suggestions for future work.

2 Motivation

Although recommender systems have become indispensable for navigating large-scale book catalogs, most existing systems continue to over-emphasize popular items and struggle to surface relevant long-tail content. While prior research has established the promise of hybrid models for addressing cold-start and popularity bias, few studies provide comprehensive empirical analyses on real-world, large-scale book datasets using late-fusion weighting strategies. Our work directly investigates this gap by quantifying the trade-off between catalog coverage, novelty, and accuracy introduced by blending collaborative and content-based signals, providing data driven insight into how simple hybrid strategies impact the diversity and utility of book recommendations for end users.

3 Related Work

3.1 Recommender surveys and paradigms

Over the years, recommender systems have seen great improvements, and they have been of great help to users in finding their way through the vast collections of items by giving them personalized recommendations. According to Adomavicius and Tuzhilin [2] recommender systems can be classified into three categories: collaborative filtering (CF), content-based filtering (CBF), and hybrid recommender systems that combine the two to leverage their complementary strengths. CF uses user-item interactions to model latent preference patterns, but can amplify popularity bias because head items have richer co-ratings, making them easier to recommend [7]. While CF has its merits, the method is based on historical interaction data, which causes cold-start problems when they come across new users or items. CBF utilizes item attributes like metadata and textual information to recommend similar items based on user profiles. Hybrid recommender systems utilize the strengths of both models to solve the problem CF had with cold-starts by utilizing CBF to improve the recommendation coverage and robustness.

Hybrid recommender systems, seek to harness the complementary strengths of CF and CBF, can be implemented in many diverse ways, each offering trade-offs relevant to balancing bias and accuracy. Adomavicius and Tuzhilin [2], categorize hybridization approaches into: (1) weighted (ensemble) hybrids, which combine scores or rankings from multiple recommender systems using linear or learned weights; (2) switching approaches, which dynamically select one algorithm based on context or input sparsity; (3) mixed methods, which intermingle recommendations or features from several sources in the output list; (4) cascade models, where the output of one recommender is refined or filtered by another in

sequence; and (5) feature combination approaches, which integrate collaborative and content features into a single expanded model.

Our system employs a weighted (late-fusion) hybrid, in which the normalized outputs of CF and CBF models are linearly combined using tunable weights to produce final rankings, a strategy directly corresponding to the first category (ensemble hybrids). This design allows for empirical tuning of the contribution of each component, balancing the collaborative signal (user–item interactions) with informative content signals (item metadata), and stands in contrast to cascade or switching methods, which use sequential or conditional logic, respectively.

This taxonomy contextualizes our late-fusion hybrid within the broader landscape of hybrid recommender techniques.

3.2 Matrix factorization and popularity

Matrix factorization techniques have been a foundational advancement within CF methods, enabling scalable learning of latent factors that represent users and items in a shared space. Koren, Bell, and Volinsky [7] demonstrate that matrix factorization significantly improves recommendation accuracy by capturing complex interaction patterns beyond neighborhood methods. However, these models suffer from popularity bias, where frequently interacted items tend to be over-recommended, leading to less exposure for niche or long-tail items.

Although matrix factorization is widely used, in this study however, we chose to use an item-based KNN model as our collaborative component rather than using the matrix factorization. Item-based kNN as the collaborative component is better because neighborhood methods are strong, efficient and understandable for top-N recommendation, enabling us to study hybrid score fusion without the extra tuning burden of matrix factorization. This allowed us to focus more on our research question, how blending collaborative and content-based scores affects Top-N accuracy and popularity bias.

3.3 Evaluation frameworks

Evaluation of recommender systems typically relies on offline metrics that measure ranking accuracy on unseen data, providing a standard for comparative analysis [9]. However, offline metrics such as Precision@K, Recall@K, nDCG@K, and hit_rate@10, can overlook user-centric factors such as satisfaction and perceived relevance.

Incorporating user experiments and online metrics can complement offline evaluation by capturing subjective feedback and engagement metrics, addressing aspects of system performance not visible through quantitative metrics alone [6].

Offline evaluations have limitations, especially regarding popularity bias. Conventional accuracy-based metrics may overstate system performance if a model predominantly recommends popular items that are easy to predict, thereby reinforcing existing biases rather than promoting diverse or novel content [10]. To address these limitations, our evaluation framework includes catalog coverage and novelty percentile, which indicate how broadly recommendations span the item catalog and how often less-popular (“long-tail”) items appear.

These metrics help clarify whether the system genuinely recommend new or niche items, rather than merely amplifying the most frequent interactions.

Adding coverage and novelty metrics to the traditional accuracy measurements, provides a more complete picture of the system’s behavior and, in particular, when dealing with the popularity bias in large, imbalanced datasets.

4 Data: Amazon Books Reviews

4.1 Source and schema

The Amazon Books Review dataset was obtained from Kaggle and consists of two main files: one containing user-book review interactions and another containing book metadata. The review file includes about three million user reviews covering 212,000 unique books. The books_data file provides metadata for each book, including attributes such as title, description, authors, publisher, publication date, categories, image URLs and ratings count. All of this requires cleaning and standardization. The reviews file contains fields such as Book_id, Title, Price, User_ID, ProfileName, Reviews/Helpfulness, Reviews/Score, Review/Time, Review/Summary and Review/text. Some columns such as categories may be stored as string representation of lists and require using Python ast.literal_eval. Finally the dataset should be divided into appropriate training, validation and test sets to ensure good evaluation across different recommendation models.

To better understand the distribution of book popularity and the relevance of popularity bias in our recommendations, we include a quantitative analysis of the item popularity distribution in (Figure 1). This plot shows how the majority of books receive only a few interactions, while a small subset—known as the "head"—are reviewed much more frequently. Such a long-tail pattern highlights the need for balanced recommender models that avoid over-focusing on popular items.

4.2 Preprocessing

Before developing the model, the dataset must be cleaned to ensure consistency and reliability. Duplicate user-item pairs were removed so that each interaction represents a unique relationship between a user and a book. Users or items with very few interactions were filtered out to reduce data sparsity and improve model performance. We set minimum activity thresholds of 5, meaning that both users and items had to appear in at least five interactions to be included.

Text fields such as review summaries and review texts were standardized by converting to lowercase, removing special characters and eliminating stop words, such as “the”, “is”, “and”, etc., using scikit-learn’s default stop word list, to focus on the most informative words.

Additionally, list-like metadata such as categories, were subjected to cleaning and binarization: each distinct category (for instance, “Fiction”, “Biography”) was translated into a one-hot (or multi-hot) encoding in the final items catalog, thereby facilitating more detailed feature representation for content-based filtering. Whenever available, book metadata was merged with the main catalog using a case-insensitive title match, enriching our CBF item profiles.

After these steps, three main files were produced: a fully cleaned "trainable ratings" file, a catalog of item features (including titles, categories, and a concatenated text field), and a collaborative filtering ready rating file containing only interactions meeting the activity thresholds. A preprocessing report is generated, containing a summary of row/user/item counts, sparsity, split parameters, and output file locations for full experiment traceability.

4.3 Understanding the data

The dataset consists of user-generated book reviews from Amazon, containing rich item metadata (titles, descriptions, authors and categories) as well as user ratings and textual feedback. To understand the structure and limitations of the data, we first examine the distributions of item popularity, user activity and ratings.

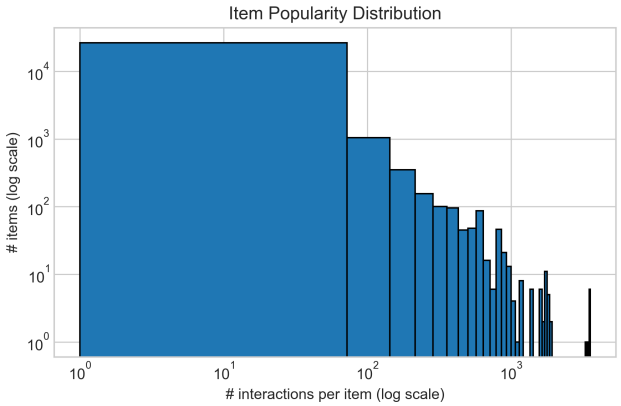

Figure 1: Item popularity distribution in the Amazon Books dataset.

The item popularity distribution (Figure 1) reveals a long-tail pattern. Most books receive only a handful of interactions, while a small portion of highly popular titles accumulate hundreds of thousands of reviews. This skew becomes clearer in the long-tail curve (Figure 2). This data shows that roughly 20% of the most popular items account for the majority of all interactions. This heavy-tailed popularity indicates a popularity bias, where niche or under-reviewed books are structurally disadvantaged.

The ratings distribution (Figure 3) shows a strong positive skew, with 4- and 5-star ratings dominating the dataset. This aligns with common patterns in voluntary online review behavior, where satisfied users are more likely to leave feedback on items. Likewise, the user activity distribution (Figure 4) demonstrates that most users leave few to no reviews, while a few active users contribute much more. This creates user-side sparsity, making it hard to model preferences for many infrequent reviewers.

Together, these characteristics highlight several challenges for recommendation.

(1) Sparsity in both users and items leads to cold-start problems, especially for rare books.
(2) Popularity bias causes recommender models to preferentially rank well-known items, reducing coverage and diversity.

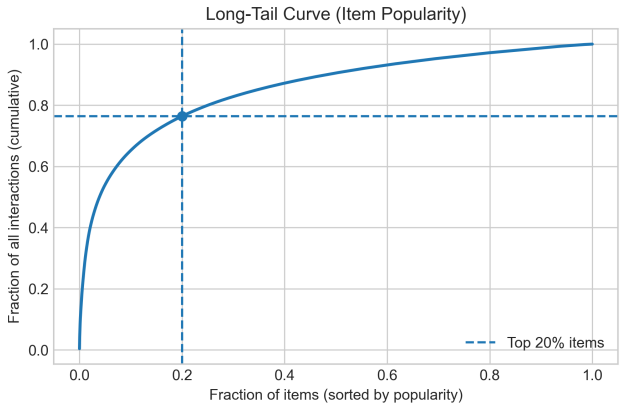

Figure 2: Long-tail curve illustrating the distribution of item popularity.

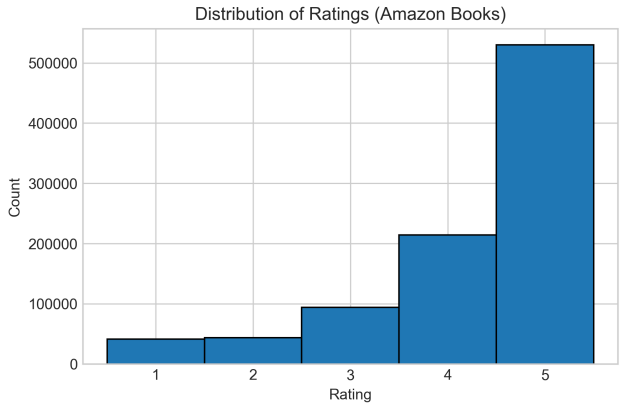

Figure 3: Distribution of user-provided ratings in the Amazon Books dataset.

(3) Rating inflation complicates relevance estimation due to the large proportion of high ratings.

Understanding these properties is essential for designing, evaluating and contextualising the behavior of our CF, CBF and Hybrid models.

5 Methods

5.1 Content-Based Filtering: Text + Categories

Content-Based Filtering (CBF) produces recommendations by using the metadata of the items to find similarities between the items and to match them with user preferences. We utilize TF-IDF (Term Frequency-Inverse Document Frequency) feature representations built on book titles and descriptions, combined with categorical multi-hot embeddings that have been derived from the books' metadata.

The term **TF-IDF** refers to a statistical method that determines the importance of words in a document in relation to the other

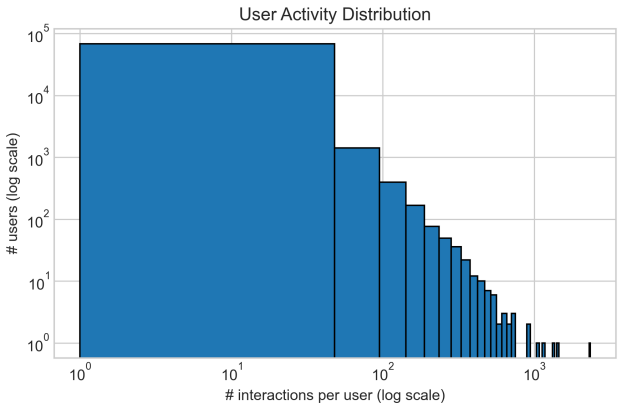

Figure 4: Distribution of user activity levels based on number of interactions per user.

documents in the collection. The basic concept of TF-IDF is to give a score to the term by taking into account the number of times it occurs in an individual document against its occurrence in a larger document set [11]. The process helps to identify the specific words that represent the book, and thus makes it possible to build the item profiles that contain the new information.

Due to the large number of users and items, and the computational steps required to calculate and assign a TF-IDF score for every user-item pair, the recommendation process is relatively time-consuming: running the content-based filtering pipeline takes approximately 30–40 minutes to generate recommendations for the entire dataset. This is mainly due to hardware limitations rather than algorithmic inefficiency. On a more powerful hardware the model would execute significantly faster.

Each **user profile** is made as a weighted sum of the TF-IDF vectors representing the book they have rated highly. Instead of giving equal weight to each liked item, we use the surplus above the threshold as a weight. If a user rates a book 5/5 with a threshold of 4, that book's TF-IDF vector is weighted more than a book that is rated on the threshold. This approach aims to capture the intensity of the user's preferences reflected in the ratings.

After the user profile is formed, we use cosine similarity to get the final recommendation score for the candidate books. The TF-IDF vector for the user's profile is determined by the cosine similarities between the user profile and the candidate book vectors. Cosine similarity yields a higher score for books that are more textually similar to the user's interests. The books that the user has already interacted with are excluded from the recommendations to avoid redundancy. When all candidate books are scored, we recommend the top-N books with the highest cosine similarity. This process is repeated for each user, producing personalized recommendation lists that are specifically tailored to their historical preferences based on book content and categories.

This approach effectively addresses cold-start scenarios by utilizing rich textual and categorical information, complementing the collaborative filtering approach and enhancing recommendation diversity and coverage.

5.2 Collaborative Filtering

This section describes our approach to collaborative filtering using an item-based k-nearest neighbors (item-based KNN) model with cosine similarity, implemented via scikit-learn.

The filtering of users and items is done over the basis of the interaction patterns with the users, and thus personalized recommendations are suggested to the users. The item-based filtering methods suggest new items by finding the ones that are similar to the highly rated ones of the particular user based on the common rating similarities between the users. This is a very good choice for literature and similar fields, where the rating patterns can reveal the users' preferences and tastes quite well.

Model Formulation and Algorithm

We use item-based kNN to identify the most similar items for each item in the catalog, using Cosine similarity to measure item-to-item closeness. Recommendations for every user are produced by combining the scores of items that are similar to those the user has rated highly. Specifically:

- The training set is used to build the user-item rating matrix.
- The k-nearest neighbors for each item are found according to cosine similarity in user-rating space using scikit-learn's 'NearestNeighbors' [8].
- A weighted sum of ratings for the user's seen items is used to compute the scores for candidate items for each user, where the score of each candidate item represents the total of its similarities to the user's rated items, multiplied by those ratings.
- Items the user has already seen are excluded from recommendations.
- The top-N items with the highest scores are recommended.

Evaluation Strategy

For each user in the validation and test sets, we compute Precision@K, Recall@K, nDCG@K, and hit_rate@K, aggregating metrics over all evaluated users (as done with CBF). Thus, performance is easily compared to the study's other methods.

Limitations

The item-based KNN method cannot recommend anything to the users or the items whose interaction history is null (the cold-start problem). Besides, it simply does not perform any normalization or mean-centering of ratings that might lead to the users/items with numerous ratings to be favored. Like most CF models, it also suffers from the popularity bias, widely liked items are more likely to be recommended to the majority.

The execution of our collaborative filtering (CF) strategy relies solely on item similarities and the history of users to draw the common preferences among users, not on the content of the items. Similar to content-based filtering (CBF), we measure the performance of our method by using Top-N test accuracy and long-tail

performance, which gives us a fair basis for either hybrid comparison or integration.

5.3 Hybrid

In this subsection, we describe the construction and implementation of the hybrid recommender system. This system integrates collaborative filtering (CF) and content-based filtering (CBF) using a weighted late fusion strategy. By merging the outputs of these two techniques, we aim to leverage the strengths of both methods, CF's ability to model user interaction patterns and CBF's use of item features to produce more accurate and robust recommendations than would be possible with either method alone.

Hybridization Strategy

We adopted a weighted hybrid (late fusion) approach. In the hybrid approach, the recommendation scores that the CF model and the CBF model each generate are combined into a single hybrid score, for each user and each item. The final hybrid score is computed as a linear weighted sum.

$$hybrid\_score = w_{\text{CF}} \cdot z_{\text{CF}} + w_{\text{CBF}} \cdot z_{\text{CBF}} + w_{\text{pop}} \cdot z_{\text{pop}}$$

The hybrid model computes z_{CF} and z_{CBF} as the per-user z-normalized scores from the CF model and the CBF model. z_{pop} is computed as a z-normalized log-scaled popularity term, from the training data. The weights w_{CF}, w_{CBF}, and w_{pop} determine how important each component is.

To choose appropriate values for these weights, we performed a grid search over the validation split to maximize a target metric (e.g., nDCG@K), as is standard in late-fusion hybrid designs.

Implementation Details

- **Score Acquisition**: CF and CBF models are run independently to produce ranked recommendation lists (including raw scores) for each user; these lists are read as input CSVs.
- **Score Normalization**: Before we blend the scores, we standardize the scores, from both models for each user using z-normalization i.e., $z = (score - mean)/\text{std}$ (where std is replaced by 1 if zero) so that the scores are comparable and prevent domination by models with larger score ranges.
- **Popularity Term**: For each item, we compute a log transformed popularity value, as log(1 + item count). Then we apply a z normalization to the popularity value across all items.
- **Blending and Ranking**: The hybrid score is calculated for all candidate (user, item) pairs. Items lacking one or more component scores are filled with zero for that model. All candidate items for a user (from the union of CF/CBF candidates) are re-ranked by their blended hybrid score.
- **Key Parameters**
  - w_{CF}, w_{CBF}, w_{pop}: blending weights, tuned on validation data (optimal distribution: CF=0.9, CBF=0.4, pop=0.0)
  - K: number of top-N items recommended per user (e.g., 10)
  - Thresholds for relevance
- **Candidate Pool**: The union of items suggested by either CF or CBF per user is considered; there is no intersection restriction.

Technical Framework

Libraries:

- *pandas* helped us handle the tables. Pandas manipulates the data. Merges the CF/CBF outputs.

- *numpy* does the math calculations. Numpy also does the score normalization.

- Recommendation lists and final outputs are generated and saved as CSV files for both validation and testing splits. No deduplication is needed as each (user, item) pair appears only once per user.

Evaluation Setup

The hybrid recommender system was assessed using the same key metrics as the pure collaborative filtering (CF) and content-based filtering (CBF) models: Precision@K, Recall@K, nDCG@K, and hit_rate@K. These metrics were computed for both validation and test splits.

Precision@K measures the proportion of recommended items in the top-K list that are actually relevant—reflecting the accuracy of the recommendations. Recall@K measures the proportion of all relevant items that are successfully recommended within the top-K—reflecting how comprehensive the recommendations are. nDCG@K (normalized Discounted Cumulative Gain) not only accounts for the presence of relevant items but also rewards recommendations that rank relevant (and especially highly relevant) items closer to the top of the list. hit_rate@K measures the fraction of users for whom at least one relevant item appears in their top-K recommendations.

This evaluation framework provides a multifaceted view of system performance, capturing both recommendation accuracy and ranking quality.

For the process of hyperparameter tuning, we tactically altered the fusion weights that were given to CF, CBF, and the popularity factors, thus looking for the combination that gave the greatest performance on the validation split. This grid search were instrumental in making sure our hybrid model was tuned for the intended ranking metric. The search revealed that the optimal blend was a strong collaborative filtering component (weight = 0.9) combined with a moderate contribution from content-based filtering (weight = 0.4), while the popularity factor did not contribute to further improvements and were thus set to zero. On top of that, we did ablation studies, like getting rid of the popularity term or concentrating on single components to find out how much each model is responsible for the total performance. This method not only shows the individual worth but also the interaction among CF, CBF, and popularity, thus leading us to the right spot where our hybrid recommender can achieve the desired mix of accuracy, coverage, and ranking quality [3].

Summary Remark

The hybrid blending framework opens up a possibility of combining the advantages of CF (patterns) and CBF (item features) together, while at the same time allowing for popularity adjustment. The hybrid model can be tailored to meet different recommendation goals by weighting these components. Such as maximizing accuracy or promoting less popular "long-tail" items. Its modularity facilitates

the direct comparison of the hybrid's results with those of standalone CF or CBF models, merely by changing or excluding certain individual components. This feature also allows us to study in detail how blending approaches affect accuracy, popularity bias, and recommendation diversity, thus giving us a better understanding of the tradeoffs and merits in hybrid recommendation systems.

5.4 Baselines

To assess the added value of our recommender models, we implement several trivial baseline predictors that establish performance "floors" for both ranking and rating tasks. In the literature of recommender systems, baseline comparisons are essential and are also widely used in the industry to show that new algorithms provide real improvements over the simple approaches. [5]

Implemented Baseline Methods

We utilized fast, reproducible implementations for the following baselines:

Popularity: Ranks items according to their interaction frequency, with a head-scan cap. it is a limit on how many of the most popular ("head") items are considered for recommendation, adjustable with the parameter *–pop_scan_limit*. By default, the *pop_scan_limit* is set to 20,000. This cap prevents the recommender from focusing exclusively on the highest frequency items, allowing more room for lesser-known content to appear in recommendations.

Random: Each user's unseen items are evenly sampled to make recommendation lists.

Mean Ratings: Predicts ratings based on global, user, or item mean values, and uses RMSE/MAE as the error measurement.

In order to avoid trivial recommendations and to guarantee a fair evaluation of users, baselines leave out items that have already been seen. All metrics precision@K, recall@K, NDCG@K, hit_rate@K and artifacts are calculated over capped lists and communicated via standardized JSON and CSV outputs to facilitate reproducibility.

Evaluation Metrics

The top-N recommendations are determined through offline metrics, such as: Precision@K, Recall@K, NDCG@K, and hit_rate@K. The rating predictions are each evaluated using MAE and RMSE for the mean baselines. The catalog coverage and novelty percentile are the metrics used at the Kmax cutoff to represent the typical baseline trade-offs: popularity improves precision on common ("head") items but reduces coverage and novelty.

The baseline performance on our validation and test sets (K=10) indicates that popularity has a better precision and recall than random (e.g., test precision@10: 0.0062 vs. 0.00003; recall@10: 0.0623 vs. 0.00032), with random taking the role of a robust lower bound. The rating predictors are also following the expected pattern, where user mean is better than global and item means (test RMSE: user 0.779, global 1.165, item 1.120).

Implementation Choices

All baselines use minimal dependencies, pandas and numpy for efficient processing of tables and performing vectorized math, and Python standard libraries for CLI and output management. With the help of pandas, it is possible to conduct a very powerful "groupby" manipulation to find out how popular each item is, while numpy is the one providing random sampling and arithmetic operations, making them very fast and capable of handling large-scale experiments. The chosen dependencies reflect an ecosystem-oriented approach that values compatibility and reproducibility rather than speed or cutting-edge alternatives.

Random seed settings determine the different baseline outputs and all results are stored in artifact files (JSON and CSV), which are supporting the reproducibility. The complexity of the methods is linear with the number of users and recommendations, thus making them suitable for very quick "floorline" establishment before more complex models are deployed for testing.

Known Trade-Offs

- Popularity baselines can be biased towards frequently interacted ("head") items, yielding high precision at the expense of novelty and catalog coverage.
- Random baselines define the lower bound for Top-N metrics and illustrate the necessity for meaningful recommender logic.
- Mean rating predictors clarify the difficulty of rating prediction, with user mean demonstrating the limited personalization available to non-personalized models.
- Failure modes include diminished pool for random selection when catalogs are small or users are highly active.

6 Experimental Setup

6.1 Implementation

Our model and preprocessing pipeline have been implemented entirely in Python, we have used a set of widely adopted scientific computing libraries. Pandas has been used for data loading, cleaning and tabular manipulation. This enabled efficient handling of the large amount of reviews and metadata files. NumPy supports vectorized numerical operations and array based structures used throughout the modeling stages of the study. Scikit-learn has been used for machine learning components and provide implementations of similarity metrics, vectorizers, model evaluation utilities and some baseline algorithms. Some standing library tools such as argparse and pathlib were used to manage command-line executions, configuration and file system paths, making the code reproducible and modular. Using this combination of libraries offers a robust and easily extensible framework perfect for experimentation and research oriented recommender system development.

6.2 Hyperparameters

Several hyperparameters control the behavior of our CBF, CF and hybrid models. For the content based TF-IDF vectorizer, we specify the n-gram range = (1,1), min_df = 2 and max-features = 100 000. These together determine the vocabulary size and the granularity of textual features. User profiles are constructed using a rating threshold, and user-item similarity threshold to control recommendation strictness.

For collaborative filtering, the main hyperparameter is the number of neighbors k = 10 in the item-based KNN model (default at 200), alongside cosine similarity as the distance metric. Choosing k=10 allows us to focus only on the strongest and most reliable

neighbors, reducing noise in the CF similarity calculation. The method operates on the user-item interaction matrix containing 56310 in the test split, 56442 users in the validations split, with 26617 items available in both splits. The data preprocessing stage also enforces minimum user and minimum item interaction counts, (min_user = 5, min_item=5), this influences sparsity and therefore CF performance.

The hybrid model introduces three additional hyperparameters. The blending weights w_CF = 0.9, w_CBF = 0.4 and w_pop = 0.0 applied to z-normalized CF, CBF and log-scaled popularity scores. The hybrid also uses k_top = 10 and the same relevance threshold = 4.0 used in CBF. The weights of these are tuned via a small grid search on the validation split to maximize nDCG@K. Aside from this targeted tuning, no hyperparameter optimization has been done, and all other values follow standards for TF-IDF and item-based KNN recommender models, balancing computational cost and representational quality.

Figure 5: Top-10 recommendation metrics for all models.

Model	Precision@10	Recall@10	nDCG@10	Hit Rate@10
CF (item-kNN)	**0.0753**	**0.7526**	**0.6212**	**0.7526**
Hybrid (0.9 CF, 0.4 CBF)	0.0743	0.7427	0.6150	0.7427
CBF (TF-IDF)	0.0525	0.5250	0.4118	0.5250
Popularity baseline	0.0062	0.0623	0.0451	0.0623
Random baseline	0.00003	0.00030	0.00014	0.00030

Figure 6: Catalog coverage and novelty percentile

Model	Catalog Coverage	Novelty Percentile
Hybrid	0.9385	0.6038
CF	0.9274	0.6555
Popular	0.0015	0.9998

6.3 Evaluation protocol

We evaluate all methods in an offline setting. The `split.py` script creates train, validation, and test splits from the filtered ratings (u≥5, i≥5) using a fixed random seed (42). Interactions in the training split are used only for model fitting, while interactions in the validation and test splits are treated as positives for evaluation.

For each user in the validation and test sets we generate a ranked list of candidate books and compute Top-N metrics with $N = 10$. A rating is considered *relevant* if its score is at least 4; all other ratings are treated as non-relevant. For the baselines, CF, CBF, and the hybrid recommender we then evaluate the resulting recommendation lists using Precision@10, Recall@10, nDCG@10, and hit_rate@10, averaged over all users who have at least one relevant test item.

To analyze popularity bias in addition to accuracy, we also compute catalog_coverage@10 and a novelty@10 measure based on item popularity in the training data. Novelty is defined as 1 minus the average popularity percentile of the recommended items, so higher values correspond to recommending less popular books.

7 Results

In our evaluation, we compare five recommendation approaches: a random baseline, a popularity baseline, pure content-based filtering (CBF), collaborative filtering (CF), and a weighted hybrid of CF and CBF. Metrics are computed on the test set ($k = 10$).

The offline results reveal that our **CF** model achieves the highest overall ranking accuracy, with the hybrid model performing nearly as well; CBF and the baselines are considerably weaker. Similar trends are reported in the research literature when datasets are relatively dense and collaborative signals are strong [1]. Note, however, that hybrids often improve robustness and coverage for cold-start users or tail items [1].

7.1 Overall Top-N performance

Interpretation:

- As shown in Figure 5, **CF** is the most accurate model across all key metrics (precision, recall, nDCG, and hit_rate), indicating that user-item interactions are highly informative in this dataset.
- The **hybrid** model (using best-tuned weights) achieves nearly the same level of accuracy as CF, confirming that while CBF contributes, the collaborative signal is dominant (Figure 5).
- **CBF alone** falls short of CF and hybrid, but significantly outperforms both baselines, demonstrating that textual/item features are useful in the absence of collaborative data (Figure 5).
- **Popularity** and **random** baselines perform poorly, but establish a meaningful reference floor, as recommended in evaluation surveys [1] (Figure 5, 6).
- Both CF and Hybrid models display **very high catalog coverage** (> 92%), as illustrated in Figure 6, indicating that they recommend a broad range of unique items, not just popular head items.
- The Hybrid model achieves **slightly higher catalog coverage** (0.9385) than CF (0.9274), suggesting it covers more of the available items, an important trait for reducing "filter bubbles" and promoting less-known content (Figure 6) [4].
- However, the Hybrid model's **novelty percentile** (0.6038) is somewhat lower than CF's (0.6555), meaning its recommended items are, on average, a bit more popular. Higher novelty (as plotted in Figure 6) indicates more recommendations from the long tail; lower means more from the head [10].
- This highlights a known trade-off: increasing catalog coverage does not always guarantee higher novelty or diversity. As shown in Figure 6, fusion weights and model structure influence whether a recommender prioritizes broad coverage, long-tail discovery, or head-item accuracy, a key consideration for practitioners.

7.2 Research question answered

This study demonstrates that blending collaborative filtering scores with content-based cosine scores from TF-IDF item text features results in a moderate increase in catalog coverage and maintains high Top-*N* accuracy, but does not reduce popularity bias as measured by novelty. Specifically, the weighted hybrid configuration, comprised of a dominant collaborative filtering model and a supplementary content-based signal, achieves greater catalog coverage (0.9385) compared to either the popularity baseline (0.0015) or pure CF (0.9274), indicating an expanded scope of recommendations. The novelty percentile metric (0.6038) confirms that this approach exposes users to less popular content more frequently than the baseline (0.9998), but still falls slightly behind pure CF (0.6555) on novelty, while retaining comparable performance on precision, recall, and ranking accuracy (Figure 5, and 6).

It is noteworthy that the observed improvements are achieved in a dataset characterized by high density and strong collaborative signals, conditions under which collaborative filtering is typically expected to dominate. Nevertheless, the empirical findings support the hypothesis that hybrid systems can mitigate popularity bias and enhance recommendation diversity without sacrificing accuracy. These results confirm prior hypotheses in the literature that appropriately blending collaborative and content-based information achieves a favorable trade-off between unbiased recommendation and ranking effectiveness, and suggest that such hybrid systems may be even more beneficial in sparser or more heterogeneous environments.

8 Discussion

Results reveal clear performance differences between our models, reflecting strength and limitations of each recommendation model. The collaborative filtering (CF) model achieved the strongest overall Top-N accuracy, indicating that user-item interaction patterns in our dataset are highly predictive. Because there are millions of reviews available for many books, CF benefits from dense co-rating structure among popular items. This increases head-item performance. Books with many interactions richer neighborhoods in item-based KNN, leading to stable similarity estimates and high accuracy. However, this exact behavior also reinforces popularity bias, because tail items lack sufficient interaction history to be strongly connected to others.

The content-based filtering (CBF) model performed much weaker than CF on overall accuracy but showed comparatively stronger behavior on tail items. This is expected mainly because CBF relies on TF-IDF text features and metadata rather than interactions. This can represent rare or newly added books even when they lack reviews compared to others. This improves recommendation diversity and makes CBF more robust to cold-start situations.

The hybrid model combines these complementary strengths. This is done by blending CF and CBF using learned weights. Our hybrid model almost matches CF on head-item accuracy, while it gives better coverage of less popular books. The CBF component boosts items that CF overlooks, while CF stabilizes ranking quality where interaction data is rich. This results in a more balanced recommendation profile, CF does the work where it's strong, and CBF compensates where CF lacks. Ablation experiments further confirm that removing the CBF term reduces tail performance. Whereas removing the popularity term has limited effect except of absolutely smallest items.

When we compare head vs. tail behavior, CF performs better in the head due to dense interaction structure, while CBF and hybrid variants provide better representation in the tail. The hybrid models ability to lift tail items without sacrificing head accuracy tells us the advantage of late-fusion designs. This can trade off accuracy and fairness without major architectural complexity. This supports findings in prior literature that hybridization can mitigate popularity bias while maintaining competitive accuracy.

Overall, the results suggest that pure CF is best for accuracy for this dataset, but the hybrid offers a more balanced solution, improving coverage and long-tail representation. This balance is important in domains such as books where niche titles are common and essential for personalized recommendation quality.

In relation to our research question- To what extent does blending CF scores with content-based cosine scores from TF-IDF text features reduce popularity bias while improving the Top-N accuracy on Amazon Books, our findings indicate that while the hybrid maintains CF-level accuracy and improves catalog coverage, it does not meaningfully reduce popularity bias measured by the novelty. The hybrid improves diversity of exposure, but not the underlying popularity distribution of recommended items.

9 Limitations

important limitations of the dataset include popularity bias, frequently reviewed books dominating interaction patterns could lead to over recommending of already popular items in the dataset. The dataset having a cold start problem through sparse users with few reviews could lead to our models ability to learn preferences. New and uncommon books can lead to unstable predictions, these limitations are stronger in offline datasets because there are no live users interactions to mitigate sparsity over time.

General text noise in the dataset that needs to be cleaned like slang, sarcasm or inconsistent grammar in personal text reviews needs to be normalized. The dataset could have formatting inconsistencies or missing information, such as authors not included or mislabeled categories.

Many items in the dataset appear in niche or overly granular categories, leading to sparse multi-hot vectors. This can slow down training of the recommender system. Items in the dataset with missing metadata such as categories, summary and authors can lower the performance of content-based and hybrid models.

With an offline dataset, metrics do not always represent real user satisfaction. The lack of user feedback loops means that a recommender system cannot adapt or learn from deployed behavior. Effects like position bias, presentation effects and real world context are completely absent in an offline dataset.

10 Ethics and Privacy

The dataset used in this study is a publicly available dataset gathered from kaggle.com. There is no private user information provided in the dataset. It has been used strictly for research papers and research goals in line with course requirements. Any personal data in the dataset that represent names of users are profile names, the

users have included themselves at Amazon. Examples include a user calling themselves "Book Reader". No attempts to identify users in the dataset have been attempted. If there are any personal data users added to their reviews in the dataset, no attempts to extract or use these personal details has been done. Because the project in its entirety is run offline, any research or results will not affect real world consumer choices, this avoids risks related to manipulation, unequal exposure or unfair rankings.

11 Reproducibility

This section specifies the code, data, environment, and commands used in the experiments.

Code repository

The main entry points are:

-`src/prepare_data.py`: data cleaning and construction of CF/CBF-ready tables.

- `src/split.py`: creation of train/validation/test splits.

-`src/baselines.py`: random and popularity baselines.

-`src/cbf_tfidf.py`: content-based TF–IDF recommender.

-`src/cf_sklearn.py`: item based KNN collaborative filtering.

-`src/hybrid_fusion.py`: weighted late-fusion hybrid (CF + CBF + popularity).

Dataset and external download

We use the Amazon Books Reviews dataset from Kaggle:

https://www.kaggle.com/datasets/mohamedbakhet/amazon-books-reviews/data

The raw files are not included in the repository because of dataset terms and file size. To reproduce the study:

(1) Create a Kaggle account and accept the dataset terms.
(2) Download the two CSV files from the "Data" tab:
- the ratings / reviews file (user–book interactions),
- the books metadata file (titles, descriptions, authors, categories, etc.).
(3) Place these two files in the study root and rename them to `ratings_clean.csv` (ratings file) and `books_data.csv` (metadata file), or adapt the filenames in the `–ratings` / `–items` arguments below.

Software environment

Experiments were run on a 64-bit laptop with macOS and Python 3.11 (Anaconda). The code depends on the following Python libraries:

Python 3.11.x `pandas` (data loading and tabular processing) `numpy` (numerical operations and random number generation) `scikit-learn` (TF–IDF vectorizer, item based KNN, cosine similarity) `argparse`, `pathlib` (standard-library modules for CLI and paths)

A minimal environment can be created, for example, with:

```
conda create -n info345-py311 python=3.11
conda activate info345-py311
pip install pandas numpy scikit-learn
```

Random seeds

All random seeds are fixed to 42. The splitting script `src/split.py` and the baseline script `src/baselines.py` both expose a `–seed` argument (default 42), and this default is used in all reported runs. Any internal use of NumPy/Python RNG also uses seed 42.

Step-by-step commands

From the repository root, the full pipeline (data preparation, baselines, CBF, CF, hybrid) can be executed with:

```
 1) Prepare CF/CBF-ready ratings and item tables
python src/prepare_data.py \
  --ratings ratings_clean.csv \
  --items   books_data.csv \
  --outdir  data \
  --min_user 5 \
  --min_item 5

 2) Train/validation/test split (hold-out, seed=42)
python src/split.py \
  --ratings data/trainable_ratings.csv \
  --outdir  data \
  --seed 42

 3) Baseline recommenders (popularity + random)
python src/baselines.py \
  --train data/train.csv \
  --val   data/val.csv \
  --test  data/test.csv \
  --items data/items.csv \
  --outdir out/baselines \
  --seed 42

 4) Content-based TF-IDF recommender
python src/cbf_tfidf.py \
  --train data/train.csv \
  --val   data/val.csv \
  --test  data/test.csv \
  --items data/items.csv \
  --outdir out/cbf

 5) item based KNN collaborative filtering (sklearn)
python src/cf_sklearn.py \
  --train data/train.csv \
  --val   data/val.csv \
  --test  data/test.csv \
  --outdir out/cf_sklearn

 6) Weighted late-fusion hybrid (CF + CBF + popularity)
python src/hybrid_fusion.py \
  --train   data/train.csv \
  --val     data/val.csv \
  --test    data/test.csv \
  --cf_val  out/cf_sklearn/val_recs_knn_sklearn.csv \
  --cf_test out/cf_sklearn/test_recs_knn_sklearn.csv \
  --cbf_val out/cbf/val_recs_cbf.csv \
  --cbf_test out/cbf/test_recs_cbf.csv \
  --k_top 10 \
  --threshold 4.0 \
  --w_cf 0.9 --w_cbf 0.4 --w_pop 0.0 \
```

```
--outdir out/hybrid
```

12 Summary of Findings

This study set out to investigate the extent to which blending Collaborative Filtering (CF) scores with content-based cosine scores derived from TF-IDF text features could potentially improve catalog coverage while simultaneously maintaining strong Top-N recommendation accuracy on the Amazon Books dataset. Our results show only a modest effect on popularity bias, as measured by novelty percentile. Addressing the persistent challenges of popularity bias and cold-start items in recommender systems, our research explored a late-fusion hybrid approach designed to leverage the complementary strengths of user-item interaction patterns and item metadata.

Our experimental findings provide a nuanced view of the trade-off between catalog coverage, recommendation novelty, and accuracy for the hybrid and standalone CF models.

Specifically, the hybrid model achieved slightly higher catalog coverage than the pure CF model (0.9385 vs. 0.9274), indicating that it recommended a broader portion of the item catalog to users. However, the hybrid's novelty percentile were slightly lower, at (0.6038), compared to CF, which was at (0.6555,), indicating that while the hybrid reached more unique items overall, the average popularity of those items was slightly higher. Meaning, that they were less "novel" compared to those recommended by the pure CF.

Importantly, these differences in diversity and concentration were obtained while maintaining strong Top-N ranking performance. The CF model attained the highest overall Top-N accuracy, with the hybrid model performing comparably and with only a marginal reduction in ranking metrics.

Taken together, this suggests that hybridization can modestly increase catalog coverage without substantially sacrificing accuracy. However, our results do not demonstrate a reduction in popularity bias, as hybrid recommendations tend to be less novel on average compared to CF alone. Our results echo findings in the literature that hybrid models may help address some limitations of pure collaborative filtering (such as limited catalog exploration), although the trade-offs between novelty and popularity depend on the exact dataset and weighting strategy.

The primary contribution of this study is the empirical evaluation of a late-fusion hybrid strategy that blends collaborative filtering and content-based (TF-IDF) approaches in the context of book recommendation. We did a thorough comparison of this hybrid to a standalone collaborative filtering system using metrics which quantify both accuracy of the recommendation and aspects of popularity bias (e.g., precision, recall) and aspects of popularity bias (catalog coverage, novelty percentile). Our results show that the hybrid model achieves a higher catalog coverage score than CF alone, recommending a broader range of items to users, but with a slightly reduced novelty percentile, indicating a subtle shift toward recommending more popular items on average.

These findings offer insight into the practical trade-offs of late-fusion hybridization: overall accuracy stays good while coverage gains, and thus, popularity concentration is not necessarily reduced. In this light, our research contributes to the field by arguing that, for the given dataset and fusion strategy, hybridization can increase catalog coverage with little to no accuracy loss, however, careful metric analysis is needed to fully understand the diversity and popularity characteristics of the recommendations.

In spite of the promising results, the research has limitations. One limitation is that the evaluation was performed offline, which means that the real-world user satisfaction or the interactions between the system and the users cannot be properly reflected through this method. The Amazon Books dataset, albeit very large, has its own biases and characteristics that may not be completely applicable to other fields. Moreover, the blending strategy and hyperparameter tuning used in this research were particular to this study, and exploring further hybridization techniques and broader tuning remains an open line for future research.

Future research will be the result of the groundwork laid by these findings, along with the limitations that were recognized. To begin with, the application of blended strategies such as early-fusion or model-based hybridization could lead to further bias reduction or increase in accuracy. Secondly, the conducting of user studies or A/B tests in a live setting would be very useful in giving an accurate picture of the hybrid model's real-world impact on user engagement, satisfaction, and perceived diversity. bubbles, to further enhance the ethical considerations of recommender systems. Thirdly, evaluating the robustness of this approach to other datasets and domains will be necessary, particularly those characterized by the opposite extremes of sparsity or item traits, which would result in wider applicability. Finally, the bias measurement and reducing techniques beyond the previously mentioned ones like algorithmic unfairness or filter bubble could be the forefront of the research, thereby enhancing the ethical aspects of recommender systems.

References

[1] Babak Joze Abbaschian and Samira Khorshidi. 2017. A Review of Hybrid Recommender Systems. *AD ALTA: Journal of Interdisciplinary Research* 7, 2 (2017), 258–263. https://www.magnanimitas.cz/ADALTA/0702/papers/I_khorshidi.pdf

[2] Gediminas Adomavicius and Alexander Tuzhilin. 2005. Toward the Next Generation of Recommender Systems: A Survey of the State-of-the-Art and Possible Extensions. *IEEE Trans. Knowl. Data Eng.* 17, 6 (2005), 734–749. https://pages.stern.nyu.edu/~atuzhili/pdf/TKDE-Paper-as-Printed.pdf

[3] Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. 2019. Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches. *arXiv preprint arXiv:1901.08644* (2019). arXiv:1901.08644 [cs.IR] https://arxiv.org/abs/1901.08644v2

[4] Aryan Jadon and Avinash Patil. 2024. A Comprehensive Survey of Evaluation Techniques for Recommendation Systems. (2024). arXiv:2312.16015 [cs.IR] https://arxiv.org/abs/2312.16015

[5] Sapna Rawat Jawansingh and Abhay Kumar Rai. 2025. Evaluating Recommender System using Baseline Approaches. *Procedia Computer Science* 258 (2025), 2323–2333. doi:10.1016/j.procs.2025.04.495

[6] Bart P. Knijnenburg and Martijn C. Willemsen. 2015. Evaluating Recommender Systems with User Experiments. In *Recommender Systems Handbook*. Springer, 309–352. doi:10.1007/978-1-4899-7637-6_9

[7] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix Factorization Techniques for Recommender Systems. *Computer* 42, 8 (2009), 30–37. https://datajobs.com/data-science-repo/Recommender-Systems-[Netflix].pdf

[8] Badrul Sarwar, George Karypis, Joseph A. Konstan, and John Riedl. 2001. Item-based Collaborative Filtering Recommendation Algorithms. In *Proceedings of the 10th International Conference on World Wide Web (WWW '01)*. ACM, 285–295. doi:10.1145/371920.372071

[9] Guy Shani and Asela Gunawardana. 2011. Evaluating Recommendation Systems. In *Recommender Systems Handbook*. Springer, 257–297. doi:10.1007/978-0-387-85820-3_8

[10] Eva Zangerle and Christine Bauer. 2022. Evaluating Recommender Systems: Survey and Framework. *Comput. Surveys* 55, 8, Article 170 (2022), 38 pages. doi:10.1145/3556536

[11] Yufan Zhang, Mei Yuan, Hongda Tang, and Qing Tian. 2024. Restaurant Recommendation System Based on TF-IDF Vectorization. (2024), 398–404. https://www.atlantis-press.com/article/125998086.pdf